Winget & Microsoft Store App Management with Microsoft Intune

Implementation Solution Guide

Version 1.0.0 · 2026-07-21

Contents

Executive Summary — Winget & Microsoft Store App Management with Microsoft Intune 4

What you get . . . 4

What this delivers . . . 4

The deployment model . . . 4

A note on accuracy . . . 4

Outcome . . . 5

1. Solution Overview — Winget & Microsoft Store App Management with Microsoft Intune 5

What this is . . . 5

How it works . . . 5

Required vs. Available (the distinction that matters) . . . 6

Choose the right lane . . . 6

Benefits . . . 6

Limitations (be honest with buyers) . . . 7

The standard deployment model . . . 7

2. Requirements Guide — Winget & Microsoft Store App Management with Microsoft Intune 7

2.1 Licensing . . . 7

2.2 Device enrollment and the Intune Management Extension (IME) . . . 8

2.3 The Microsoft Store app (new) type is available in your tenant . . . 8

2.4 Company Portal deployed (for self-service / Available apps) . . . 8

2.5 The exact app catalog listing and package identifier . . . 8

2.6 Supported Windows baseline . . . 8

2.7 Winget prerequisites present on the device . . . 9

2.8 Pilot and broad user groups . . . 9

2.9 Government cloud differences (if applicable) . . . 9

Checklist . . . 9

Global Requirements — Intune Solutions (category-level) 10

G1. Microsoft Entra ID tenant . . . 10

G2. Microsoft Intune subscription . . . 10

G3. Administrative roles . . . 10

G4. Device enrollment in Intune . . . 10

G5. Pilot security group . . . 10

G6. Supported Windows baseline . . . 10

G7. Network access to Microsoft cloud endpoints . . . 10

3. Implementation Guide — Winget & Microsoft Store App Management with Microsoft Intune 11

Named objects (reuse these names verbatim everywhere) . . . 11

Step 1 — Create the pilot and broad groups . . . 11
Step 2 — Add a Microsoft Store app (the Required baseline app) . . . 11
Step 3 — Assign the Store app as Required to the pilot ring . . . 12
Step 4 — Add a winget long-tail app as Available (self-service) . . . 12
Step 5 — (If the app is winget-community-only, not in the Store) wrap the winget command . . . 12
Step 6 — Confirm the update path for each app . . . 13
Step 7 — Validate on a pilot device . . . 13
Step 8 — Expand to the broad ring . . . 13
5. Testing Guide — Winget & Microsoft Store App Management with Microsoft Intune 14
Test 1 — Required Store app installs automatically . . . 14
Test 2 — A winget long-tail app installs . . . 14
Test 3 — An Available app self-installs via Company Portal . . . 14
Test 4 — An update applies . . . 14
Test 5 — Assignment and self-service are configured correctly . . . 15
Test 6 — Fleet report . . . 15
Pass/fail gate . . . 15
6. Troubleshooting Guide — Winget & Microsoft Store App Management with Microsoft Intune 15
A. A Required Store app is not installing . . . 15
B. A winget app won't detect, or shows the wrong version . . . 16
C. A self-service (Available) app isn't appearing in Company Portal . . . 16
D. Updates are lagging . . . 17
E. Uninstall behavior isn't what you expected . . . 17
F. Scripts . . . 18
G. When to escalate . . . 18
7. Best Practices — Winget & Microsoft Store App Management with Microsoft Intune 18
8. Deliverables Summary — Winget & Microsoft Store App Management with Microsoft Intune 19
Documentation . . . 20
PowerShell scripts . . . 20
What the buyer receives . . . 20
Outcome . . . 21
Appendix A — PowerShell Scripts 22

Executive Summary — Winget & Microsoft Store App Management with Microsoft Intune

White-label · Intune Solutions

Most of the apps your users ask for every day are mainstream, publicly available Windows apps — a PDF reader, a media player, a browser, common utilities. You should not have to hand-package each one, and your users should not have to open a ticket to get one installed. Microsoft Intune deploys these two ways: **Microsoft Store apps** (the modern, **winget-backed** Store app type, drawing on a large public catalog) and **winget** for the long tail — with **assignment**, **updates**, and **Company Portal self-service**. This is a complete, **deploy-it-today** package for standing up Store and winget app management end to end, written for an administrator with no prior experience.

What you get

• A plain-language **Solution Overview** of Microsoft Store (new) apps, winget for apps beyond the Store, Required vs. Available (self-service), updates, and how to **choose the right lane** versus the Enterprise App Catalog (curated prepackaged apps) and custom Win32.

• A **Requirements Guide** with verify / implement-if-missing for every prerequisite, including winget prerequisites and Company Portal.

• An **Implementation Guide** of precise, click-by-click Intune steps in the required order, built around two worked example apps — one Required, one self-service.

• Five **PowerShell scripts** for discovery, PASS/FAIL validation, install/update reporting, install testing, and a `-WhatIf`-gated assignment change.

• **Testing** and **Troubleshooting** guides for a confident rollout.

What this delivers

A working Store + winget app-management practice — mainstream apps deployed **Required** for the baseline, published **Available** in Company Portal for self-service, kept current through the service-managed update path, and rolled out **pilot-first** so a bad assignment never reaches the whole fleet at once.

The deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand.** The pilot ring is the gate.

A note on accuracy

Microsoft's app catalog, the new Store app type, and update behavior change over time; this guide describes the model confidently and flags the tenant-specific specifics (catalog availability, exact navigation, update cadence, Government-cloud differences) to **verify against current Microsoft documentation** before you promise them to users.

Outcome

Mainstream Windows apps that install, self-serve, and update through Intune — without hand-packaging the apps your users ask for every day — deployable by a single administrator and validated before fleet rollout.

1. Solution Overview — Winget & Microsoft Store App Management with Microsoft Intune

What this is

Most of the apps your users ask for every day — a PDF reader, a media player, a browser, common utilities — are mainstream, publicly available Windows apps. You should not have to hand-package each one. Microsoft Intune deploys these two ways, and this solution stands both up end to end:

- **Microsoft Store apps (the "new" Store app type)** — Intune's modern Store app type is

winget-backed: it installs apps from the Microsoft Store catalog, which draws on the public **Windows Package Manager (winget)** community catalog. You search for the app by name, add it, and Intune handles install and update. This gives you a **large public catalog** of Windows apps without building a single `.intunewin`.

- **Winget for the long tail** — apps beyond what you deploy from the Store are covered by winget:

either they surface in the new Store app search (add them the same way), or, for winget-community apps that are not exposed through the Store, you wrap the winget command as a Win32 app (see the cross-reference below). The exact set of apps available through each path, and how each updates, is a moving target — **verify against current Microsoft documentation** for your tenant.

On top of both, Intune adds **assignment** (who gets the app), **self-service** (users install on-demand from **Company Portal**), and **updates** (keeping installed apps current).

How it works

1. In the **Microsoft Intune admin center**, you add a **Microsoft Store app (new)** by searching the

catalog for the app by name; Intune records the app's package identifier and its metadata.

1. You **assign** the app. **Required** assignments install automatically on the targeted users'

devices; **Available for enrolled devices** publishes the app in **Company Portal** so users install it themselves (self-service). An **Uninstall** assignment removes it.

1. For an app you cannot or do not want to source from the Store, you deploy it with **winget** — the

supported route for winget-community-only apps is a **Win32 app** that runs the `winget` install command (packaging is covered by the sibling *Intune Win32 Application Packaging & Deployment* product). Whether a given app is available directly as a new-Store app is **verify-with-Microsoft**.

1. **Updates:** the new Store (winget-backed) app type is designed to keep apps current automatically

as the publisher ships new versions; a winget-wrapped Win32 app updates when you deploy a new version or run a winget upgrade. **The exact update cadence and which path auto-updates vs. needs management is verify-with-Microsoft** — do not assume; confirm for your tenant.

1. You **pilot** every app on a small ring, validate install + self-service + update, then **expand**

to the fleet.

Required vs. Available (the distinction that matters)

- **Required** — Intune installs the app automatically on the assigned users' or devices' machines.

Use it for baseline apps everyone must have (e.g. the corporate PDF reader).

- **Available for enrolled devices** — the app appears in **Company Portal** and the user installs it

on demand. Use it for the long tail of apps some people want and others do not — this is **self-service**, and it removes the "please install X for me" ticket.

- **Uninstall** — Intune removes the app from the assigned targets.

Choose the right lane

You need…	Use	Why
A mainstream public app (reader, browser, media player)	**Microsoft Store app (new)** — this product	Winget-backed, large public catalog, no packaging, auto-update
A **curated, prepackaged** enterprise app with vendor-maintained detection/update	**Enterprise App Catalog** — see *Enterprise App Catalog Deployment in Microsoft Intune*	Microsoft-curated prepackaged Win32 apps with managed updates
A **winget-community-only** app not in the Store, or any in-house / custom installer	**Custom Win32** — see *Intune Win32 Application Packaging & Deployment*	Full control of a `.intunewin`, install/uninstall, detection
A deployment that is **failing** (install error, wrong state, detection)	**Break/fix** — see *Intune App Deployment Troubleshooting & Failure Remediation (Win32/LOB/Store)*	Structured diagnosis and remediation

This product covers the **Store + winget public-catalog + self-service** lane. It **cross-references** and does **not** duplicate the Enterprise App Catalog (curated, prepackaged) or the custom Win32 packaging products.

Benefits

- **No packaging for mainstream apps** — add from a public catalog by name; no `.intunewin` build.
- **Self-service** — Company Portal turns "install X for me" tickets into one user click.
- **Automatic updates** for Store/winget apps (confirm cadence per tenant — verify-with-Microsoft).
- **One assignment model** — Required, Available, Uninstall, targeted by ring, same as every app.

• **Safe rollout** — pilot ring first, so a bad assignment never hits the whole fleet at once.

Limitations (be honest with buyers)

• **Catalog coverage varies.** Not every app is in the Store/winget public catalog, and which apps are is a moving target — **verify-with-Microsoft** before promising an app to users.

• **Update behavior is not something you fully control.** Auto-update timing for Store/winget apps is service-managed; the precise cadence and any per-app exceptions are **verify-with-Microsoft**.

• **Winget-community-only apps** may need a Win32 wrapper (a small packaging step) rather than a direct Store add — see the custom Win32 product.

• **Self-service needs Company Portal** installed and the user signed in; without it, Available apps have nowhere to appear.

• **Version pinning is limited.** The Store/winget model favors "latest"; when you must hold a specific version, a custom Win32 package gives you that control.

The standard deployment model

> ***Add a Microsoft Store (winget-backed) app → assign it Required for baseline / Available for self-service → cover the long tail with winget → confirm updates → pilot on a ring → validate → expand to broad.*** *This is the repeatable way:* ***Assess → Configure → Pilot → Validate → Expand.*** *A bad assignment never reaches production because the pilot ring is the gate.*

2. Requirements Guide — Winget & Microsoft Store App Management with Microsoft Intune

Verify **every** requirement before deploying. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites are in `global-requirements-intune-solutions.md`.

Throughout this solution we use two **worked example apps** so every rule is concrete:

• A **Microsoft Store app** the whole team needs — **Adobe Acrobat Reader** (publisher **Adobe Inc.**), deployed **Required**.

• A **winget long-tail app** some users want — **VLC media player** (publisher **VideoLAN**), deployed **Available** for self-service.

Replace these names with your own apps. Confirm each app's **exact catalog listing and package identifier** in your tenant before you deploy — **verify against current Microsoft documentation**, because catalog contents and identifiers change.

2.1 Licensing

Description: An Intune-bearing license (Intune Plan 1). Store and winget app management is a built-in Intune capability. **Why:** Intune stores the app assignment, installs via the Intune Management Extension, and reports install/update state. **Verify:** M365 admin center → **Billing** → **Licenses**; confirm an Intune plan is assigned to the users you will target. **Implement:** Assign M365 E3/E5 or Business Premium (Intune included), or standalone Intune Plan 1.

2.2 Device enrollment and the Intune Management Extension (IME)

Description: Targeted Windows devices enrolled in Intune, with the IME present and healthy. **Why:** Store (new) / winget-backed apps are installed by the IME on enrolled devices. **Verify:** Intune → **Devices** → **All devices** lists the device; run `WINGETSTORE-Discovery.ps1` (checks for the `Microsoft Intune Management Extension` service on-device context and lists deployed apps). **Implement:** Enable automatic enrollment and enroll the device. The IME installs automatically once a Store/winget or Win32 app or PowerShell script is assigned to the device.

2.3 The Microsoft Store app (new) type is available in your tenant

Description: The **Microsoft Store app (new)** app type (winget-backed) in **Apps** → **Windows** → **Add**. **Why:** This is the type that deploys public-catalog apps without packaging; the older "Microsoft Store app (legacy)" type is deprecated. **Verify:** Intune → **Apps** → **Windows** → **Add** → **App type** dropdown lists **Microsoft Store app (new)**. **Exact type name and its availability are verify-with-Microsoft** — the portal label and behavior change over time. **Implement:** No action if present. If absent or renamed, confirm the current app type name and its prerequisites in Microsoft's current documentation for your cloud (Commercial vs Government may differ — see §2.9).

2.4 Company Portal deployed (for self-service / Available apps)

Description: The **Company Portal** app installed on targeted Windows devices and users signed in. **Why:** **Available** (self-service) app assignments only appear to users **inside Company Portal**; without it there is nowhere for an on-demand install to surface. **Verify:** On a pilot device, open **Company Portal** and confirm it launches and shows the signed-in user's assigned apps. **Implement:** Deploy Company Portal (itself a Microsoft Store app) to your devices, or confirm it is delivered via your enrollment configuration. **The exact recommended delivery method is verify-with-Microsoft** for your tenant.

2.5 The exact app catalog listing and package identifier

Description: For each app you deploy, the **exact publisher and the package/catalog identifier** Intune resolves when you search — so you add the right app and not a look-alike. **Why:** Public-catalog search returns multiple near-identical results; the exact publisher + identifier is what guarantees you deploy (and later detect/report) the correct app. **Verify:** In **Apps** → **Windows** → **Add** → **Microsoft Store app (new)** → **Search**, confirm the result's **publisher** matches (e.g. **Adobe Inc.** for Acrobat Reader, **VideoLAN** for VLC) before selecting. Record the identifier Intune shows. **Implement:** If unsure which listing is authoritative, confirm the publisher and package identifier against **current Microsoft / winget documentation** — do not guess from the display name alone.

2.6 Supported Windows baseline

Description: Windows 10/11 on a currently supported build with recent updates. **Why:** The new Store app type and winget support depend on modern Windows components. **Verify:** `winver` on a pilot device, or Intune → device → **Hardware**. **Implement:** Deploy **Update rings** or upgrade to a supported build. **The exact minimum Windows build required for the new Store app type is verify-with-Microsoft.**

2.7 Winget prerequisites present on the device

Description: The **Windows Package Manager (winget)** components (App Installer) present on targeted devices for winget-backed installs. **Why:** Winget-backed Store apps and any winget-wrapped Win32 apps rely on the winget/App Installer runtime being available in system context. **Verify:** On a pilot device, confirm **App Installer** is present (Store → Library, or check for the `winget` command). **Whether Intune requires you to deploy App Installer explicitly, or it is present by default, is verify-with-Microsoft. Implement:** If missing, deploy **App Installer** (a Microsoft Store app) to the fleet before relying on winget installs. Confirm the current supported approach in Microsoft's documentation.

2.8 Pilot and broad user groups

Description: Microsoft Entra security groups for the pilot users and the broad fleet. **Why:** Every app is piloted before fleet rollout; Available (self-service) apps target **users**, so user groups are the natural scope. **Verify:** Entra → **Groups**; the pilot and broad groups exist. **Implement:** Create them per Implementation Step 1 (or run `WINGETSTORE-Set-Assignment.ps1`, which can reference existing groups). Name them `Store & Winget Apps – Pilot – Ring 1` and `Store & Winget Apps – Broad – Ring 2`.

2.9 Government cloud differences (if applicable)

Description: Government tenants (GCC / GCC High / DoD) can differ from Commercial for app catalog availability, the new Store app type, and portal URLs. **Why:** The Store/winget public catalog and the new Store app type may have different availability, timing, or endpoints in US-sovereign clouds; a Commercial procedure can be wrong for Government. **Verify:** Confirm the app type, catalog availability, and admin portal URL (e.g. `intune.microsoft.us` vs `intune.microsoft.com`) for your cloud. **Implement: Verify against Microsoft's US-sovereign-cloud documentation.** Do not assume the Commercial catalog/behavior applies to GCC High / DoD.

Checklist

- Intune licensing assigned to targeted users
- Devices enrolled; IME healthy — `WINGETSTORE-Discovery.ps1`
- Microsoft Store app (new) type available in the tenant *(verify-with-Microsoft)*
- Company Portal deployed for self-service apps
- Exact publisher + package identifier confirmed per app *(verify-with-Microsoft)*
- Supported Windows baseline confirmed
- Winget / App Installer prerequisites present *(verify-with-Microsoft)*
- `Store & Winget Apps – Pilot – Ring 1` and `– Broad – Ring 2` groups created

- Government-cloud differences checked (if applicable) *(verify-with-Microsoft)*

Global Requirements — Intune Solutions (category-level)

These requirements apply to **every** solution in the **Intune Solutions** category. They are published once at the category level so individual listings stay small. Confirm these before deploying *any* Intune solution. Each entry: **Description · Why · Verify · Implement-if-missing.**

G1. Microsoft Entra ID tenant

Description: An active Microsoft Entra ID (Azure AD) tenant. **Why:** It is the identity backbone for every cloud and Intune-managed resource. **Verify:** Sign in to `https://entra.microsoft.com` as an administrator. **Implement:** Create a Microsoft 365/Azure subscription, which provisions a tenant.

G2. Microsoft Intune subscription

Description: An Intune-bearing license plan. **Why:** Intune delivers all configuration, compliance, and security policies. **Verify:** `https://intune.microsoft.com` loads; **Tenant administration** → **Tenant status** shows active. **Implement:** Assign Microsoft 365 E3/E5, Business Premium, or standalone Intune Plan 1.

G3. Administrative roles

Description: Appropriate Entra/Intune admin roles for the operator. **Why:** Creating policies and enrollment settings requires elevated rights. **Verify:** Entra → **Roles and administrators**; confirm **Intune Administrator**. **Implement:** Assign **Intune Administrator**; reserve Global Administrator for tenant-wide changes.

G4. Device enrollment in Intune

Description: Targeted Windows devices are enrolled and manageable. **Why:** Policies only reach enrolled devices. **Verify:** Intune → **Devices** → **All devices**; the device appears and is **compliant**. **Implement:** Enable **Automatic Enrollment** and join devices to Entra ID.

G5. Pilot security group

Description: A Microsoft Entra security group for pilot/targeted users or devices. **Why:** Every solution is piloted before fleet rollout. **Verify:** Entra → **Groups**; the pilot group exists. **Implement:** Entra → **Groups** → **New group** → **Security** → add members.

G6. Supported Windows baseline

Description: Windows 10 21H2+ or Windows 11 with current updates. **Why:** Modern Intune features assume a patched, supported OS. **Verify:** `winver`, or Intune → device → **Hardware**. **Implement:** Deploy **Update rings** or upgrade to Windows 11.

G7. Network access to Microsoft cloud endpoints

Description: Outbound HTTPS to Microsoft 365, Entra, and Intune service URLs. **Why:** Enrollment, policy sync, and authentication depend on these endpoints. **Verify:** From a device, reach `https://login.microsoftonline.com` and `https://enrollment.manage.microsoft.com`. **Implement:** Allow Microsoft's published Office 365/Intune URL & IP ranges through proxy/firewall.

The standard deployment model (all solutions): Assess → Configure → Pilot → Validate → Expand.

3. Implementation Guide — Winget & Microsoft Store App Management with Microsoft Intune

Follow these steps **in order**. Everything happens in the **Microsoft Intune admin center** (`https://intune.microsoft.com`). **Exact console navigation, blade names, and the "Microsoft Store app (new)" type label are verify-with-Microsoft** — the portal changes; confirm the current path against Microsoft's documentation. Where a path is given, it is the current expected drill-down.

> ***Model:*** *Add a Microsoft Store (winget-backed) app, assign it* ***Required*** *for a baseline app and* ***Available*** *(self-service) for a long-tail app, cover winget-only apps with a Win32 wrapper, confirm updates, assign to a* ***pilot*** *ring, validate, then expand to* ***broad****. The pilot ring is the gate.*

Named objects (reuse these names verbatim everywhere)

We use the `<Purpose> – <Platform> – <Stage/Type>` naming convention so every object is self-describing and searchable. For the two worked example apps:

Object	Name to create
Pilot user group	`Store & Winget Apps – Pilot – Ring 1`
Broad user group	`Store & Winget Apps – Broad – Ring 2`
Required Store app	`Adobe Acrobat Reader – Store – Required`
Available (self-service) app	`VLC media player – Winget – Available`
Winget-only Win32 wrapper (if needed)	`<App> – Winget – Win32 Wrapper`

Step 1 — Create the pilot and broad groups

1. Entra (`https://entra.microsoft.com`) → **Groups** → **New group** → **Group type: Security** → **Group

name = `Store & Winget Apps – Pilot – Ring 1` → **add a few representative pilot** users → Create**. (Or run `WINGETSTORE-Set-Assignment.ps1`, which references existing groups.)

1. Repeat to create `Store & Winget Apps – Broad – Ring 2` (the rest of the fleet's users). **Create**.

Step 2 — Add a Microsoft Store app (the Required baseline app)

1. Intune → **Apps** → **Windows** → **Add**.

2. **App type:** select **Microsoft Store app (new)**. **Select**.

(If the type name differs in your tenant, use the current winget-backed Store app type — verify-with-Microsoft.)

1. **Search the Microsoft Store app** → type `Adobe Acrobat Reader` → in the results, confirm the

publisher is Adobe Inc. (not a look-alike) → **Select**. Intune fills in the app's package identifier and metadata.

1. **App information:** confirm **Name** = `Adobe Acrobat Reader – Store – Required` (set the display

name to this exact value), confirm publisher, add a short description. Set **Install behavior** to **System** for a device-wide baseline app (use **User** only if the app requires per-user install). **The available install-behavior options and their exact effect are verify-with-Microsoft. Next.**

Step 3 — Assign the Store app as Required to the pilot ring

1. On the **Assignments** tab, under **Required**, **Add group** → select

`Store & Winget Apps – Pilot – Ring 1`.

1. Do **not** add the broad ring yet — the pilot is the gate. **Next** → **Review + create** → **Create.**

Step 4 — Add a winget long-tail app as Available (self-service)

1. Intune → **Apps** → **Windows** → **Add** → **Microsoft Store app (new)** → **Select**.

2. **Search** → type `VLC` → confirm the **publisher is VideoLAN** (disambiguate from re-packaged

look-alikes) → **Select**.

1. **App information: Name** = `VLC media player – Winget – Available`; confirm publisher. **Next.**

2. On the **Assignments** tab, under **Available for enrolled devices**, **Add group** → select

`Store & Winget Apps – Pilot – Ring 1`. This publishes the app in **Company Portal** for those users to install on demand. **Next** → **Review + create** → **Create.**

Step 5 — (If the app is winget-community-only, not in the Store) wrap the winget command

Some apps exist in the **winget community catalog** but do not surface as a Microsoft Store app (new). For those, deploy a **Win32 app** that runs the winget install command. **Whether a given app is directly addable as a new-Store app is verify-with-Microsoft** — check the catalog first.

1. Create a Win32 app named `<App> – Winget – Win32 Wrapper` (packaging is covered by the sibling

Intune Win32 Application Packaging & Deployment product — do not re-derive it here).

1. **Install command (example):**

`winget install --id VideoLAN.VLC --silent --accept-package-agreements --accept-source-agreements --scope machine` **Uninstall command (example):** `winget uninstall --id VideoLAN.VLC --silent` The **exact package** `--id`**, the supported flags, and machine-scope behavior in system context are verify-with-winget/Microsoft** — confirm the current id and switches before deploying.

1. Author a detection rule (file/version or registry) and assign it Required or Available as above.

Step 6 — Confirm the update path for each app

1. For **Microsoft Store app (new)** apps, updates are designed to be **service-managed** — the app

moves to newer versions as the publisher ships them, without you re-uploading. **The exact update cadence, and whether any app is excluded, is verify-with-Microsoft.**

1. For a **winget-wrapped Win32 app**, updates happen when you deploy a new version (or run a scheduled

winget upgrade). Plan which apps are auto-managed vs. version-controlled by you (see the Overview "Limitations").

1. Record, per app, whether updates are **auto** (Store/winget) or **managed by you** (Win32-pinned).

Step 7 — Validate on a pilot device

1. On a pilot device, sync policy: **Settings → Accounts → Access work or school → (account) → Info →

Sync**.

1. Confirm the **Required** app `Adobe Acrobat Reader – Store – Required` installs automatically.

2. Open **Company Portal**, find `VLC media player – Winget – Available`, and **Install** it

on-demand (self-service).

1. Run `WINGETSTORE-Validate.ps1` (PASS/FAIL) and `WINGETSTORE-Test-Install.ps1` to confirm the pilot

apps are deployed, assigned, self-service is configured, and installs detect. Complete the **Testing Guide**.

1. Run `WINGETSTORE-Report.ps1` to confirm Intune reports **Installed** for the pilot users/devices.

Step 8 — Expand to the broad ring

1. Only after the pilot passes the Testing Guide gate, edit each app's **Assignments**:

• `Adobe Acrobat Reader – Store – Required` → under **Required** → **Add group** →

`Store & Winget Apps – Broad – Ring 2`.

• `VLC media player – Winget – Available` → under **Available for enrolled devices** → **Add group** →

`Store & Winget Apps – Broad – Ring 2`. **Save.**

1. Re-run `WINGETSTORE-Report.ps1` over the next cycle and chase any **Failed** or missing installs

(cross-reference *Intune App Deployment Troubleshooting & Failure Remediation (Win32/LOB/Store)*).

Order of operations

`Pilot/broad groups → add Store app (Required) → assign pilot → add winget app (Available) → assign pilot → (winget-only? wrap as Win32) → confirm updates → validate → expand broad`

5. Testing Guide — Winget & Microsoft Store App Management with Microsoft Intune

Run after the apps apply to the pilot ring (Implementation Step 7) and again after expanding to the broad ring. The worked example apps are `Adobe Acrobat Reader – Store – Required` (Required) and `VLC media player – Winget – Available` (self-service).

Test 1 — Required Store app installs automatically

Do: On a pilot device (user in `Store & Winget Apps – Pilot – Ring 1`), sync policy, wait a cycle, then run `WINGETSTORE-Validate.ps1`. **Expected:** `Adobe Acrobat Reader – Store – Required` installs **without user action** and is present on the device. **PASS/FAIL:** PASS if the app is installed and detected; FAIL if it never installs or reports Failed.

Test 2 — A winget long-tail app installs

Do: Confirm the winget-backed app deploys on the device — either the Available app after a self-service install (Test 3) or a winget-wrapped Win32 app if you used Step 5. Check with `WINGETSTORE-Test-Install.ps1`. **Expected:** The winget-sourced app (e.g. VLC media player) is installed and detected on the device. **PASS/FAIL:** PASS if installed and detected; FAIL if absent or reporting Failed.

Test 3 — An Available app self-installs via Company Portal

Do: On the pilot device, open **Company Portal**, locate `VLC media player – Winget – Available`, and click **Install**. **Expected:** The app appears in Company Portal for the pilot user, installs on demand, and Company Portal shows it as **Installed**. **PASS/FAIL:** PASS if the app is visible in Company Portal and self-installs; FAIL if it never appears (check the Available assignment and Company Portal presence) or the install fails.

Test 4 — An update applies

Do: For a Store/winget app with a newer version available, allow the service-managed update path a cycle, then re-check the installed version with `WINGETSTORE-Test-Install.ps1` (or `WINGETSTORE-Report.ps1` for the fleet). **The expected update timing is verify-with-Microsoft** — confirm the current cadence for your tenant before treating a lag as a failure. **Expected:** The installed app moves to the newer version without re-adding the app in Intune. **PASS/FAIL:** PASS if the version

advances within the documented cadence; FAIL if it stays behind after the expected window (then see Troubleshooting E).

Test 5 — Assignment and self-service are configured correctly

Do: Run `WINGETSTORE-Validate.ps1`, which checks that the pilot Store app is assigned **Required** and the pilot winget app is assigned **Available** (self-service) to `Store & Winget Apps – Pilot – Ring 1`. **Expected:** Required app → Required assignment present; Available app → Available assignment present; both scoped to the pilot group. **PASS/FAIL:** PASS if both assignments match; FAIL if either is missing or mis-scoped.

Test 6 — Fleet report

Do: Run `WINGETSTORE-Report.ps1 -CsvPath C:\Temp\wingetstore.csv`. **Expected:** A clear per-app, per-device Installed / Failed / Not applicable / Pending breakdown across the assigned rings, exported to CSV. **PASS/FAIL:** PASS if the report generates and pilot devices show Installed; FAIL on widespread Failed.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1–5 PASS for the pilot ring `Store & Winget Apps – Pilot – Ring 1`
Broad pass	Tests 1, 3, 6 PASS for `Store & Winget Apps – Broad – Ring 2` after expansion

Do **not** assign the broad ring until the pilot gate passes.

6. Troubleshooting Guide — Winget & Microsoft Store App Management with Microsoft Intune

Use `WINGETSTORE-Discovery.ps1` to see what's deployed and how it's assigned, `WINGETSTORE-Validate.ps1` for the pilot PASS/FAIL state, `WINGETSTORE-Report.ps1` for fleet install/update status, and `WINGETSTORE-Test-Install.ps1` **on the device** to confirm a local install + detected version. The Intune Management Extension log on the device is `C:\ProgramData\Microsoft\IntuneManagementExtension\Logs\IntuneManagementExtension.log`. For deep break/fix beyond this table, use the *Intune App Deployment Troubleshooting & Failure Remediation (Win32/LOB/Store)* product.

A. A Required Store app is not installing

Symptom	Cause	Remediation
`Adobe Acrobat Reader – Store – Required` never installs on a targeted device	Assignment missing or scoped to the wrong group	Run `WINGETSTORE-Validate.ps1`; confirm the Required assignment targets `Store & Winget Apps – Pilot – Ring 1`; fix with `WINGETSTORE-Set-Assignment.ps1`
Device shows the app as Pending forever	IME not present / device not synced	Confirm enrollment; sync (**Settings** → **Accounts** → **Access work or school** → **Sync**); check the IME service with `WINGETSTORE-Test-Install.ps1`
App shows **Not applicable**	Device doesn't meet the app's requirements (OS/architecture) or the new Store app type isn't supported on that build	Confirm the supported Windows baseline (Requirement 2.6); **verify the new Store app type's minimum build with Microsoft**
Wrong app installed	A look-alike catalog listing was added	Re-add the app confirming the **publisher** (e.g. **Adobe Inc.**) and package identifier (Requirement 2.5)

B. A winget app won't detect, or shows the wrong version

Symptom	Cause	Remediation
App is on the device but Intune/report shows not installed	Detection/reporting lag, or (for a winget-wrapped Win32) a wrong detection rule	Confirm locally with `WINGETSTORE-Test-Install.ps1` (registry check is id-independent); for a Win32 wrapper fix the detection rule per the Win32 packaging product
`winget list` finds it but the wrapper reports Failed	The winget id or scope is wrong in the install command	**Verify the exact winget** `--id` **and** `--scope machine` **behavior with winget/Microsoft docs**; re-test the command in system context
Version differs from expected	Store/winget installs "latest"; you assumed a pinned version	If you must pin a version, use a custom Win32 package (see the Win32 product) rather than the Store lane

C. A self-service (Available) app isn't appearing in Company Portal

Symptom	Cause	Remediation
`VLC media player – Winget – Available` not visible to the user	The assignment intent is not **Available**, or it's scoped to devices/another group	Run `WINGETSTORE-Validate.ps1`; ensure the app is assigned **available** to the user's group; fix with `WINGETSTORE-Set-Assignment.ps1 -Intent available`
No apps show in Company Portal at all	Company Portal not installed, or user not signed in	Deploy Company Portal (Requirement 2.4); confirm the user is signed in with their work account
App appears but won't install on click	winget/App Installer prerequisites missing	Confirm App Installer is present (Requirement 2.7); **verify the required winget prerequisites with Microsoft**

D. Updates are lagging

Symptom	Cause	Remediation
Installed apps stay on an older version	Store/winget update cadence is service-managed, not instant	**Confirm the expected update cadence with Microsoft (verify-with-Microsoft)** before treating a lag as a failure; allow the documented window, then re-check with `WINGETSTORE-Report.ps1` (Version column)
A specific app never updates	It's a version-pinned Win32 wrapper, not a Store/winget app	Deploy the new version (or a scheduled winget upgrade) — pinned apps do not auto-update
Some devices update, others don't	Sync / connectivity gaps	Sync the device; confirm network access to Microsoft endpoints (Global Requirement G7)

E. Uninstall behavior isn't what you expected

Symptom	Cause	Remediation
Removing an app didn't uninstall it	No **Uninstall** assignment was added; removing the Available/Required assignment alone may not remove an installed app	Add an **Uninstall** intent assignment (`WINGETSTORE-Set-Assignment.ps1 -Intent uninstall`) to actively remove it
User re-installs a removed app	It's still **Available** in Company Portal	Remove the Available assignment (or replace with Uninstall) so it no longer offers the app

Symptom	Cause	Remediation
Uninstall leaves remnants	The publisher's uninstaller is incomplete	For thorough removal of a winget-wrapped app, use a custom Win32 uninstall command (see the Win32 product)

F. Scripts

Script	Use it to
`WINGETSTORE-Discovery.ps1`	List every Store (new) / winget app and its assignments (Required / Available / Uninstall) — read-only
`WINGETSTORE-Validate.ps1`	PASS/FAIL the pilot: the Required app + the Available (self-service) app are assigned to the pilot group — read-only
`WINGETSTORE-Report.ps1`	Export per-app / per-device Installed / Failed / Not applicable + detected version to CSV — read-only
`WINGETSTORE-Test-Install.ps1`	Confirm a local install and its version on a pilot device (IME + winget + registry) — read-only
`WINGETSTORE-Set-Assignment.ps1`	Add a Required / Available / Uninstall assignment — the only state-changer, **defaults to** `-WhatIf`

```
Get-Service -Name 'Microsoft Intune Management Extension'        # IME present/running
Get-Command winget                                               # winget/App Installer available?
Get-ItemProperty 'HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall\*',
  'HKLM:\SOFTWARE\WOW6432Node\Microsoft\Windows\CurrentVersion\Uninstall\*' |
  Select-Object DisplayName, DisplayVersion   # installed app + version (id-independent detection)
```

The IME log (`IntuneManagementExtension.log`) shows the install action and result per app — the first place to look.

G. When to escalate

If the assignment is correct, the device is enrolled, synced, and healthy, the app is confirmed in the catalog with the right publisher, and Intune still reports Failed after the documented policy/update cycle, collect the `IntuneManagementExtension.log` and open a Microsoft support case. Because catalog availability, the new Store app type, and update behavior change over time, **verify the current expected behavior against Microsoft documentation for your cloud** (Commercial vs Government may differ) before concluding it is a defect.

7. Best Practices — Winget & Microsoft Store App Management with Microsoft Intune

- **Pilot every app first, always.** A small `Store & Winget Apps – Pilot – Ring 1` catches a bad

assignment or a wrong app before it reaches `Store & Winget Apps – Broad – Ring 2` — the single most important practice. The pilot is the gate.

- **Match Required vs. Available to the app.** Make baseline apps everyone needs **Required**; make the long tail **Available** in Company Portal (self-service). This kills "install X for me" tickets while keeping the mandatory set installed automatically.
- **Confirm the exact publisher before you add an app.** Public-catalog search returns look-alikes; add the app whose publisher matches (e.g. **Adobe Inc.**, **VideoLAN**), and record its package identifier. A wrong listing deploys the wrong software.
- **Prefer the Store (winget-backed) lane for mainstream apps.** No packaging, auto-update — use it before reaching for a custom Win32 package. Reserve custom Win32 for winget-only, in-house, or version-pinned apps (see the *Intune Win32 Application Packaging & Deployment* product).
- **Use the Enterprise App Catalog for curated prepackaged apps.** When you need vendor-maintained detection and managed updates for a prepackaged Win32 app, that is the *Enterprise App Catalog Deployment* product's lane — not this one. Don't rebuild it here.
- **Know each app's update path.** Record whether an app auto-updates (Store/winget, service-managed) or is version-controlled by you (a pinned Win32 wrapper). Do not assume auto-update timing — **confirm the cadence with Microsoft** for your tenant.
- **Deploy Company Portal before relying on self-service.** Available assignments only appear inside Company Portal; without it, on-demand installs have nowhere to surface.
- **Name everything with `<Purpose> – <Platform> – <Stage/Type>`.** Self-describing names (apps, groups) keep a growing app list auditable and searchable, and let scripts reference objects reliably.
- **Target self-service apps to user groups.** Available apps follow the user into Company Portal; user scoping is the natural fit for opt-in apps.
- **Watch the report and chase laggards.** Track Installed / Failed / Not applicable with `WINGETSTORE-Report.ps1`; for failures, use the *Intune App Deployment Troubleshooting & Failure Remediation (Win32/LOB/Store)* product's structured break/fix.
- **Verify catalog and behavior specifics against Microsoft, don't assume.** App availability, the new Store app type's name, update cadence, and Government-cloud differences all change — confirm current facts for your tenant before promising them to users.

8. Deliverables Summary — Winget & Microsoft Store App Management with Microsoft Intune

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	Microsoft Store (new, winget-backed) apps, winget for the long tail, Required vs. Available self-service, updates, and the choose-the-lane guidance (vs. Enterprise App Catalog and custom Win32)
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing, including winget prerequisites and Company Portal
Global Requirements (Intune Solutions)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered Intune steps: add a Store app + assign, add a winget app, Required vs. Available + uninstall behavior, updates, pilot ring, expand
5. Testing Guide	Validation procedures with PASS/FAIL gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

PowerShell scripts

Script	What it does
`WINGETSTORE-Discovery.ps1`	Read-only inventory of Store/winget apps deployed, their assignments, and self-service availability (Microsoft Graph)
`WINGETSTORE-Validate.ps1`	PASS/FAIL check that the pilot Store app + winget app are deployed, assigned, and self-service is configured (read-only)
`WINGETSTORE-Report.ps1`	Install/update status export to CSV across the fleet (read-only)
`WINGETSTORE-Test-Install.ps1`	Read-only confirmation of a pilot install and its detection on-device
`WINGETSTORE-Set-Assignment.ps1`	The only state-changer: assigns a Store/winget app Required-or-Available to a pilot group; **defaults to** `-WhatIf`

What the buyer receives

- This implementation guide as a **Word** document and **PDF**.
- All five **PowerShell scripts** in a single ZIP.
- A repeatable process following the model: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand.**

Outcome

Mainstream Windows apps — Microsoft Store (winget-backed) apps and winget packages — that deploy and stay current through Intune, with Required baseline installs, Company Portal self-service for the long tail, and a pilot-first rollout so a bad assignment never reaches production.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

WINGETSTORE-Discovery.ps1

Purpose: Read-only discovery of Windows Store (new) / winget apps in Intune. Lists each app, its publisher, and its assignments broken down as Required / Available (self-service via Company Portal) / Uninstall — so you can see, at a glance, which mainstream apps are deployed and which are offered for self-service. Makes no changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\WINGETSTORE-Discovery.ps1 Requires the Microsoft Graph PowerShell SDK. GRAPH SCOPES DeviceManagementApps.Read.All (read app definitions + assignments) EXPECTED RESULTS A per-app summary: display name, publisher, and each assignment's intent (required / available / uninstall) with the target group id. No data is modified. .NOTES Intune Solutions — read-only. The Graph type for the new Store app is 'microsoft.graph.winGetApp'; confirm the current type/behavior against Microsoft documentation (verify-with-Microsoft). > [CmdletBinding()] param()

How to run: powershell -ExecutionPolicy Bypass -File .\WINGETSTORE-Discovery.ps1 Requires the Microsoft Graph PowerShell SDK. GRAPH SCOPES DeviceManagementApps.Read.All (read app definitions + assignments) EXPECTED RESULTS A per-app summary: display name, publisher, and each assignment's intent (required / available / uninstall) with the target group id. No data is modified. .NOTES Intune Solutions — read-only. The Graph type for the new Store app is 'microsoft.graph.winGetApp'; confirm the current type/behavior against Microsoft documentation (verify-with-Microsoft). > [CmdletBinding()] param()

WINGETSTORE-Report.ps1

Purpose: Read-only report of how each Store (new) / winget app is deploying across the fleet — Installed / Failed / Not applicable / Pending counts, and (with -CsvPath) a per-device row including the installed version so you can spot apps lagging behind on updates. Use it to gate the pilot before expanding and to chase failures. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\WINGETSTORE-Report.ps1 powershell -ExecutionPolicy Bypass -File .\WINGETSTORE-Report.ps1 ` -AppName "Adobe Acrobat Reader – Store – Required" -CsvPath C:\Temp\wingetstore.csv Requires the Microsoft Graph PowerShell SDK. GRAPH SCOPES DeviceManagementApps.Read.All (read app install summaries + device statuses) EXPECTED RESULTS A status summary per app and, with -CsvPath, a per-device CSV (device, app, install state, detected version) for reviewing update lag. No data is modified. .NOTES Intune Solutions — read-only against Graph. New Store app type = 'microsoft.graph.winGetApp'. Update cadence is service-managed; confirm expected timing against Microsoft docs (verify-with-Microsoft) before treating a version lag as a failure. > [CmdletBinding()] param([string]$AppName, [string]$CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\WINGETSTORE-Report.ps1 powershell -ExecutionPolicy Bypass -File .\WINGETSTORE-Report.ps1 ` -AppName "Adobe Acrobat Reader – Store – Required" -CsvPath C:\Temp\wingetstore.csv Requires the Microsoft Graph PowerShell SDK. GRAPH SCOPES DeviceManagementApps.Read.All (read app install summaries + device statuses) EXPECTED RESULTS A status summary per app and, with -CsvPath, a per-device CSV (device, app, install state, detected version) for reviewing update lag. No data is modified. .NOTES Intune Solutions — read-only against Graph. New Store app type = 'microsoft.graph.winGetApp'. Update cadence is service-managed; confirm expected timing against Microsoft docs (verify-with-Microsoft) before treating a version lag as a failure. > [CmdletBinding()] param([string]$AppName, [string]$CsvPath)

WINGETSTORE-Set-Assignment.ps1

Purpose: Adds a group assignment to an existing Store (new) / winget app in Intune: -Intent required -> Intune installs the app automatically on the group's devices, or -Intent available -> the app appears in Company Portal for on-demand self-service install. SAFETY: this is the only script that changes tenant state, and it DEFAULTS TO -WhatIf — it will only PREVIEW the change and print what it would do. Re-run with -WhatIf:$false to apply. Existing assignments are read first and preserved; this call adds one assignment, it does not replace the set. HOW TO RUN Preview (default, no change): powershell -ExecutionPolicy Bypass -File .\WINGETSTORE-Set-Assignment.ps1 `-AppName "VLC media player – Winget – Available"` -GroupName "Store & Winget Apps – Pilot – Ring 1" -Intent available Apply for real: powershell -ExecutionPolicy Bypass -File .\WINGETSTORE-Set-Assignment.ps1 `-AppName "Adobe Acrobat Reader – Store – Required"` -GroupName "Store & Winget Apps – Pilot – Ring 1" -Intent required -WhatIf:$false Requires the Microsoft Graph PowerShell SDK. GRAPH SCOPES DeviceManagementApps.ReadWrite.All (add the app assignment) Group.Read.All (resolve the target group id from its name) EXPECTED RESULTS In preview (default): a "What if:" line describing the assignment that WOULD be added — no change. When applied: the assignment is added and the app's assignment list is re-printed. .NOTES Intune Solutions — state-changing, ShouldProcess-gated, DEFAULTS TO -WhatIf. New Store app Graph type = 'microsoft.graph.winGetApp' (verify-with-Microsoft). > [CmdletBinding(SupportsShouldProcess, ConfirmImpact='High')] param([Parameter(Mandatory)] [string]$AppName, [Parameter(Mandatory)] [string]$GroupName, [ValidateSet('required','available','uninstall')] [string]$Intent = 'available')

How to run: Preview (default, no change): powershell -ExecutionPolicy Bypass -File .\WINGETSTORE-Set-Assignment.ps1 `-AppName "VLC media player – Winget – Available"` -GroupName "Store & Winget Apps – Pilot – Ring 1" -Intent available Apply for real: powershell -ExecutionPolicy Bypass -File .\WINGETSTORE-Set-Assignment.ps1 `-AppName "Adobe Acrobat Reader – Store – Required"` -GroupName "Store & Winget Apps – Pilot – Ring 1" -Intent required -WhatIf:$false Requires the Microsoft Graph PowerShell SDK. GRAPH SCOPES DeviceManagementApps.ReadWrite.All (add the app assignment) Group.Read.All (resolve the target group id from its name) EXPECTED RESULTS In preview (default): a "What if:" line describing the assignment that WOULD be added — no change. When applied: the assignment is added and the app's assignment list is re-printed. .NOTES Intune Solutions — state-changing, ShouldProcess-gated, DEFAULTS TO -WhatIf. New Store app Graph type = 'microsoft.graph.winGetApp' (verify-with-Microsoft). > [CmdletBinding(SupportsShouldProcess, ConfirmImpact='High')] param([Parameter(Mandatory)] [string]$AppName, [Parameter(Mandatory)] [string]$GroupName, [ValidateSet('required','available','uninstall')] [string]$Intent = 'available')

WINGETSTORE-Test-Install.ps1

Purpose: Run ON a pilot device after an assignment applies (or after a Company Portal self-service install) to prove the app is really present. Checks two independent signals, read-only: the Intune Management Extension service is present (the installer of Store/winget apps), and the app is detected locally — via winget (if available) and via the Windows uninstall registry (DisplayName + DisplayVersion) so you can see the installed version. Makes no changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\WINGETSTORE-Test-Install.ps1 -AppName "VLC" powershell -ExecutionPolicy Bypass -File .\WINGETSTORE-Test-Install.ps1 -AppName "Acrobat Reader" -WingetId "Adobe.Acrobat.Reader.64-bit" Runs locally; no Microsoft Graph or scopes required. GRAPH SCOPES None (local device queries only). EXPECTED RESULTS PRESENT with the detected DisplayName +

version if the app is installed; NOT DETECTED otherwise. The exact winget package id is verify-with-winget; the registry check works regardless of id. .NOTES Intune Solutions — read-only, local. Run in an elevated session to read machine-scope installs. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$AppName, # display-name substring, e.g. "VLC" or "Acrobat Reader" [string]$WingetId # optional exact winget id (verify-with-winget))

How to run: powershell -ExecutionPolicy Bypass -File .\WINGETSTORE-Test-Install.ps1 -AppName "VLC" powershell -ExecutionPolicy Bypass -File .\WINGETSTORE-Test-Install.ps1 -AppName "Acrobat Reader" -WingetId "Adobe.Acrobat.Reader.64-bit" Runs locally; no Microsoft Graph or scopes required. GRAPH SCOPES None (local device queries only). EXPECTED RESULTS PRESENT with the detected DisplayName + version if the app is installed; NOT DETECTED otherwise. The exact winget package id is verify-with-winget; the registry check works regardless of id. .NOTES Intune Solutions — read-only, local. Run in an elevated session to read machine-scope installs. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$AppName, # display-name substring, e.g. "VLC" or "Acrobat Reader" [string]$WingetId # optional exact winget id (verify-with-winget))

WINGETSTORE-Validate.ps1

Purpose: Confirms the pilot state from the Implementation Guide is correct: the Required Store app exists and is assigned 'required' to the pilot group, the Available (self-service) winget app exists and is assigned 'available' to the pilot group (so it appears in Company Portal). Prints a PASS/FAIL line per check. Makes no changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\WINGETSTORE-Validate.ps1 `-RequiredAppName "Adobe Acrobat Reader – Store – Required"` -AvailableAppName "VLC media player – Winget – Available" ` -PilotGroupName "Store & Winget Apps – Pilot – Ring 1" Requires the Microsoft Graph PowerShell SDK. GRAPH SCOPES DeviceManagementApps.Read.All (read app definitions + assignments) Group.Read.All (resolve the pilot group id from its name) EXPECTED RESULTS PASS for each check when the app exists and carries the expected intent to the pilot group; FAIL (with the reason) otherwise. Overall exit is informational only. .NOTES Intune Solutions — read-only. New Store app Graph type = 'microsoft.graph.winGetApp' (verify-with-Microsoft). > [CmdletBinding()] param([string]$RequiredAppName = 'Adobe Acrobat Reader – Store – Required', [string]$AvailableAppName = 'VLC media player – Winget – Available', [string]$PilotGroupName = 'Store & Winget Apps – Pilot – Ring 1')

How to run: powershell -ExecutionPolicy Bypass -File .\WINGETSTORE-Validate.ps1 `-RequiredAppName "Adobe Acrobat Reader – Store – Required"` -AvailableAppName "VLC media player – Winget – Available" ` -PilotGroupName "Store & Winget Apps – Pilot – Ring 1" Requires the Microsoft Graph PowerShell SDK. GRAPH SCOPES DeviceManagementApps.Read.All (read app definitions + assignments) Group.Read.All (resolve the pilot group id from its name) EXPECTED RESULTS PASS for each check when the app exists and carries the expected intent to the pilot group; FAIL (with the reason) otherwise. Overall exit is informational only. .NOTES Intune Solutions — read-only. New Store app Graph type = 'microsoft.graph.winGetApp' (verify-with-Microsoft). > [CmdletBinding()] param([string]$RequiredAppName = 'Adobe Acrobat Reader – Store – Required', [string]$AvailableAppName = 'VLC media player – Winget – Available', [string]$PilotGroupName = 'Store & Winget Apps – Pilot – Ring 1')